



Projeto Banco do Brasil

Documentação Parcial GrowUp

Classificação

Acadêmico / Produto Digital

**Ferramenta Inteligente Para Geração Automática De Testes
Unitários
Utilizando Inteligência Artificial**

1. Definição do Problema e Personas IA-Augmented

1.1 Problema, Solução e Valor da IA

Problema (250 Caracteres) Definição da Dor

Desenvolvedores gastam muito tempo escrevendo testes unitários manualmente, o que reduz produtividade e frequentemente resulta em baixa cobertura e inconsistência na qualidade dos testes.

Solução Proposta de Solução

Uma ferramenta inteligente que utiliza IA (DeepSeek via API e Ollama) para gerar automaticamente testes unitários a partir do código-fonte, aumentando produtividade, cobertura e padronização.

Onde a IA Gera Valor

A IA gera valor principalmente por:

Pilar de Valor	Entrega e Impacto
Geração de Conteúdo	Criação automática de scripts e roteiros de teste: Reduz o tempo de escrita manual e amplia a cobertura das verificações desde o início.
Automação	Eliminação de tarefas repetitivas: Remove o trabalho manual exaustivo, permitindo que a equipe foque em tarefas de maior valor estratégico.
Assistência Inteligente	Análise de casos de borda: Sugere cenários de erro pouco comuns que humanos costumam esquecer, aumentando a robustez do software.

1.2 Personas

Persona 1 - Ana Vidal, Desenvolvedora Júnior com foco em produtividade

Perfil	Mulher, 26 anos, desenvolvedora Júnior no Banco do Brasil em Recife.
Objetivos	Automatizar tarefas repetitivas, garantir a cobertura de código e reduzir a ocorrência de erros em sistemas críticos
Dores	Elevado tempo gasto na criação manual de testes unitários
Comportamento Digital	Busca ferramentas que se integrem ao fluxo de desenvolvimento para acelerar entregas.
Expectativa com a IA	Geração rápida de testes unitários estruturados e sugestão de mocks coerentes para agilizar o processo de desenvolvimento.

Persona 2 - Carlos Alberto, Desenvolvedor Sênior

Perfil	Homem, 34 anos, Desenvolvedor Sênior no Banco do Brasil.
Objetivos	Garantir que os sistemas possuam testes completos e identificar falhas antecipadamente para assegurar a conformidade com as regras de negócio
Dores	Testes incompletos e baixa cobertura de cenários críticos no código.
Comportamento Digital	Utiliza ferramentas de validação e monitoramento para garantir a robustez das aplicações.
Expectativa com a IA	Identificação automática de possíveis falhas no código e antecipação de cenários críticos, incluindo edge cases.

1.3 Cenários de Uso e Edge Cases

Categoria	Descrição da Categoria
Cenários de Uso	Gerar testes a partir de uma função ou classe inteira; refatorar testes existentes e gerar testes em diferentes frameworks (Jest, JUnit).
Edge Cases	Entradas nulas/indefinidas, funções sem retorno claro, código com alta complexidade ciclomática ou tipos inesperados
Cenários de Falha	Código inválido ou incompleto enviado para análise
Falhas de Serviço (IA)	Timeout na API (Ollama/Gemini) ou respostas incompletas/truncadas.
Falhas de Saída (Output)	Geração de testes incorretos que não compilam ou não condizem com a regra de negócio.

1.4 Entrevistas e Validação

1. Entrevistas e Validação de Hipóteses

Foi realizada uma escuta ativa com profissionais da área de desenvolvimento de software ao longo das reuniões de acompanhamento do projeto. Essa etapa permitiu compreender melhor o contexto real das equipes e funcionou como base para a validação das hipóteses levantadas pelo Squad 46.

2. Descoberta dos pontos de dor:

A entrevista evidenciou problemas recorrentes no ciclo de desenvolvimento, como o elevado tempo gasto na criação manual de testes unitários, a baixa cobertura de código e a dificuldade técnica em prever e implementar todos os cenários de teste possíveis.

3. Validação do problema:

A fala dos desenvolvedores confirmou que existe uma demanda real por soluções que reduzam o esforço manual. Verificou-se que o problema abordado é uma dor comum em equipes de software, demonstrando que há uma forte necessidade de automação nesse contexto.

4. Relato com humanos e agentes inteligentes:

A validação com humanos demonstrou que o uso de Inteligência Artificial contribui significativamente para a aceleração do desenvolvimento. Já no contexto dos agentes inteligentes, a análise concentrou-se na capacidade da IA (Ollama/Gemini) em automatizar tarefas repetitivas, aumentar a qualidade do software e apoiar as equipes de QA e desenvolvimento.

2. Backlog e Engenharia de Requisitos AI-First

Backlog Priorizado e Categorizado:

Épicos:

1. Geração de testes com IA:

Este épico foca na capacidade central da ferramenta em analisar o código-fonte enviado pelo desenvolvedor e transformar a lógica de programação em suítes de testes unitários. O objetivo é utilizar modelos de linguagem para compreender o comportamento das funções e gerar automaticamente a estrutura de testes, poupando tempo e garantindo que a cobertura inicial seja feita de forma ágil e eficiente.

2. Integração com APIs (Ollama/Gemini):

O segundo épico trata da inteligência híbrida do sistema, estabelecendo a comunicação entre o motor local e a nuvem. O foco está em garantir que o Ollama processe as requisições localmente para manter a privacidade, enquanto o Gemini atua como uma camada de segurança e redundância. Essa integração permite que o sistema seja resiliente, alternando entre as APIs conforme a necessidade de processamento ou disponibilidade.

3. Interface do Usuário:

Este épico abrange toda a experiência visual e funcional do desenvolvedor com a ferramenta. O foco é criar um ambiente intuitivo onde o usuário possa carregar seu código, visualizar os testes gerados e interagir com as sugestões da IA. A interface é projetada para ser leve, utilizando indicadores de carregamento e feedback em tempo real para gerenciar o tempo de resposta da inteligência artificial.

4. Validação de testes gerados:

O último épico é dedicado à confiabilidade e segurança do código produzido. Ele estabelece uma camada de verificação que analisa se os testes gerados pela IA possuem sintaxe correta e se são compatíveis com o framework Jest. Essa etapa de validação garante que o desenvolvedor receba um material pronto para execução, minimizando erros e assegurando que os critérios de aceite do projeto sejam atendidos.

2.2 Histórias de usuário com Critérios de Aceite de IA

Histórias de Usuário e Critérios de Aceite:

As histórias de usuário foram definidas para solucionar a dificuldade de desenvolvedores em manter uma alta cobertura de testes. A principal história foca no desejo do usuário de automatizar a criação de testes unitários para economizar tempo e garantir a qualidade do código. Como critério de aceitação, estabelecemos que a IA deve interpretar a lógica do código enviado e gerar um arquivo de teste funcional, incluindo mocks de dependências externas. A garantia de funcionamento é dada por uma etapa de validação que verifica se o código gerado segue as normas do framework Jest e se as regras de negócio foram respeitadas, evitando falhas manuais.

Como Validar a IA:

Para assegurar a confiabilidade, a validação da IA é realizada em tempo real. O sistema executa uma análise sintática sobre o output gerado para detectar possíveis alucinações ou códigos incompletos. Além disso, utilizamos logs estruturados para monitorar a taxa de sucesso das respostas do Ollama e do Gemini, garantindo que a ferramenta sempre entregue um código que compile e que realmente reflita os cenários de uso (caminho feliz e casos de erro) descritos na documentação do projeto.

4. Arquitetura de Software e Stack de Desenvolvimento

4.1 Escolha da Stack de Desenvolvimento e IA:

A stack tecnológica do Squad 46 foi selecionada para garantir um ambiente de desenvolvimento moderno, seguro e de baixo custo. O backend é estruturado em Node.js com TypeScript, utilizando o framework Express para gerenciar as rotas de API. Para a camada de inteligência artificial, optamos por uma estratégia híbrida: o Ollama atua como motor principal rodando localmente (garantindo que o código do usuário não saia do ambiente seguro), enquanto o Google Gemini serve como infraestrutura de fallback. Essa combinação oferece escalabilidade e independência tecnológica, priorizando o uso de soluções gratuitas e eficientes.

Modelo Gemini: gemini-2.5-flash | config.: { model: "gemini-2.5-flash" } | troca futura: alterar o valor do model.

4.2 Desenho de Arquitetura e Tratamento de Erros:

A arquitetura foi desenhada seguindo princípios modulares, onde o serviço de IA é isolado da lógica de negócio principal. Para garantir a resiliência do sistema, implementamos um mecanismo de Circuit Breaker nas chamadas de API. Caso o processamento local via Ollama apresente lentidão excessiva ou falha de resposta, o sistema redireciona a requisição automaticamente para o Gemini. No frontend, utilizamos indicadores de progresso (spinners) para gerenciar a expectativa do desenvolvedor, mantendo a interface responsiva mesmo durante análises de códigos mais complexos que exigem maior tempo de processamento.

Persistência de Logs em Substituição ao Banco de Dados:

Neste projeto, as informações geradas durante a execução dos testes não são armazenadas em banco de dados. A persistência é feita localmente no arquivo logs.txt, registrando validações, aprovações, reprovações e erros com data, horário e descrição do evento.

Com isso, o sistema mantém rastreabilidade das execuções, permite auditoria dos resultados e simplifica a implementação, sem depender de serviços externos. Assim, o arquivo de log atua como uma persistência leve e temporária para o escopo atual do projeto.

4.3 Extensão VS Code e Integração de IA

Arquitetura do Plugin e Ambiente:

Extension Host (VS Code API)	Gerencia a interface do usuário, os comandos no menu de contexto (clique direito) e a captura do texto selecionado no editor.
Ollama (Local LLM) / Gemini API	Atua como motor de processamento de linguagem natural, recebendo o código e retornando a otimização ou o teste unitário gerado.
Node.js Runtime	Ambiente de execução da extensão, responsável pela lógica de comunicação, tratamento de strings e preparação dos dados antes do envio à IA.

Design da Extensão e Fluxo de Dados Completo:

1. Entrada	O desenvolvedor seleciona um bloco de código TypeScript/JavaScript no VS Code, clica com o botão direito e escolhe uma função, como "Generate Unit Test with BB".
2. Processamento de Contexto	A extensão identifica o escopo do arquivo e verifica se existe um servidor local do Ollama ativo ou se deve utilizar a Gemini API por meio de chave de acesso configurada.
3. Integração IA	O plugin monta o payload com o System Prompt, contendo instruções de especialista em Jest/Testing Library, junto com o trecho de código selecionado, realizando a requisição via Axios ou Fetch para o serviço de IA.
4. Injeção de Código	Após receber o retorno em Markdown/Code, a extensão faz o parsing da resposta, remove marcações desnecessárias e cria um arquivo .test.ts ou injeta o código otimizado diretamente abaixo da seleção.

Tratamento de Exceções:

Seleção vazia	Se o comando for acionado sem código selecionado, a extensão exibe um Warning Toast no canto inferior direito: "Nenhum código selecionado para análise."
Indisponibilidade de serviço	Se o Ollama não estiver rodando em localhost:11434 ou a API Key estiver inválida, o plugin captura o erro e sugere: "Verifique sua conexão ou se o serviço de IA está ativo."
Timeout de geração	Para blocos de código muito extensos, a extensão encerra a tentativa para não travar o processo principal do VS Code (Extension Host).

Resultado esperado: a extensão conecta o fluxo do desenvolvedor diretamente à IA, permitindo gerar testes, otimizar trechos de código e manter a experiência dentro do próprio VS Code, com fallback e mensagens de erro claras.

5. Relato do processo e Engenharia de Prompt

5.1 Repositório de Prompts (Prompt Ops):

Item	Detalhe
Objetivo	Instruir a IA a atuar como um assistente especializado na geração automática de testes unitários para desenvolvedores.
Contexto informado	Código-fonte da aplicação e requisitos funcionais para que a IA compreenda as regras de negócio e a lógica do sistema.
Saída esperada	Código de teste limpo, funcional e totalmente compatível com o framework Jest, seguindo os padrões técnicos do projeto.
Problemas encontrados	Dificuldades com a dependência de contexto e ocorrência de testes inconsistentes. Foram feitos ajustes para evitar alucinações.

5.2 Relato de Processo

O desenvolvimento conduzido pelo Squad 46 focou na aplicação de Inteligência Artificial para solucionar o gargalo da criação manual de testes unitários. O uso da tecnologia impactou o projeto positivamente, permitindo a automação de tarefas repetitivas, o aumento da produtividade da equipe e uma melhoria significativa na cobertura de testes. Ao utilizar o Ollama para execução local e o Gemini como suporte, o grupo conseguiu validar que a abordagem AI-First é viável e eficiente para elevar a qualidade do software e reduzir falhas em produção.

Durante o processo, as principais dificuldades enfrentadas envolveram a dependência de contexto para a geração de respostas assertivas e as limitações técnicas dos modelos locais. Para superar esses desafios, foram realizados ajustes focados na utilização eficiente do fallback e na estruturação de prompts mais precisos. O resultado final consolidou-se como uma solução coerente e alinhada aos padrões acadêmicos, demonstrando que, embora exija curadoria técnica, a IA atua como um apoio fundamental para as equipes de desenvolvimento e qualidade.